

EVALUACION PARCIAL

Java Persistence API (JPA)

Repositorios y ABM de Categorías y Productos

1. Objetivo General

Extender el proyecto Gradle del TP de la Unidad 8 creando los repositorios para las entidades Categoría y Producto, e implementando un menú de consola que permita realizar operaciones ABM sobre dichas entidades. Se incluye además una consulta JPQL personalizada para filtrar productos por categoría.

2. Archivos a Crear

El alumno debe agregar al proyecto los siguientes archivos nuevos. No se debe modificar ninguna clase del TP base.

Archivo a crear	Responsabilidad
<code>BaseRepository.java</code>	Repositorio generico <T> con operaciones CRUD comunes: guardar, buscarPorId, listarActivos, eliminarLogico.
<code>CategoriaRepository.java</code>	Extiende BaseRepository<Categoria>. Sin metodos adicionales.
<code>ProductoRepository.java</code>	Extiende BaseRepository<Producto>. Agrega buscarPorCategoria(Long id) con JPQL.
<code>Main.java</code>	Clase principal con menu de consola que expone el ABM de Categoría y Producto, y la consulta JPQL.

Esquema de paquetes

Respetar la siguiente estructura dentro de src/main/java/:

```
com.tp.jpa/  
    model/          <- ya existe (no modificar)  
    model/enums/    <- ya existe (no modificar)  
    util/           <- ya existe, contiene JPAUtil.java  
    repository/      <- NUEVO: crear este paquete con los 3 repositorios  
    Main.java        <- NUEVO: clase principal con el menu
```

3. Consigna Tecnica

3.1 BaseRepository<T> (repositorio generico)

Crear la clase abstracta BaseRepository<T> en el paquete repository. Debe recibir la Class<T> por constructor y obtener el EntityManagerFactory desde JPAUtil. Implementar los siguientes metodos:

1. guardar(T entity): persiste o actualiza la entidad usando merge(). Abre y cierra su propio EntityManager. Maneja la transaccion con begin/commit y rollback en caso de error y devuelve T.
2. buscarPorId(Long id): retorna Optional<T> usando find(). Retorna Optional.empty() si no existe.
3. listarActivos(): retorna List<T> con los registros cuyo campo eliminado = false. Usa JPQL.
4. eliminarLogico(Long id): busca la entidad por ID, establece eliminado = true y persiste el cambio. Retorna boolean indicando si encontro el registro.

Cada metodo debe abrir su propio EntityManager al inicio y cerrarlo en un bloque finally.

3.2 CategoriaRepository

Crear la clase CategoriaRepository que extienda BaseRepository<Categoria>. El constructor debe llamar a super(Categoria.class). No requiere metodos adicionales: hereda todo el CRUD del repositorio base.

3.3 ProductoRepository

Crear la clase ProductoRepository que extienda BaseRepository<Producto>. Ademas del CRUD heredado, implementar:

5. buscarPorCategoria(Long categoriaId): retorna List<Producto> con los productos activos de esa categoria. La consulta debe escribirse en JPQL con un parametro nombrado, filtrar por eliminado = false, y retornar un List<Producto>.

3.4 ABM de Categorías en Main

Implementar en la clase Main un submenu de Categorías con las siguientes opciones:

6. Alta: solicitar nombre y descripción, crear la instancia, persistir y mostrar el ID generado.
7. Baja lógica: solicitar el ID, marcar eliminado = true. Si no existe, mostrar mensaje de error.
8. Modificación: solicitar el ID, mostrar valores actuales, permitir editar nombre y/o descripción, persistir.
9. Listado: mostrar todas las categorías activas con ID, nombre y descripción.

3.5 ABM de Productos en Main

Implementar en Main un submenu de Productos con las siguientes opciones:

10. Alta: listar categorías activas para seleccionar, solicitar nombre, precio, descripción y stock, persistir.
11. Baja lógica: solicitar el ID del producto. Si no existe o ya está dado de baja, mostrar error.
12. Modificación: solicitar el ID, mostrar valores actuales, permitir editar nombre, precio y stock.
13. Listado: mostrar todos los productos activos con ID, nombre, precio, stock y nombre de su categoría.

3.6 Consulta JPQL en el menu

Agregar en Main la opción "Productos por categoría" dentro de un submenu de Reportes. Al seleccionarla:

- Listar las categorías activas para que el usuario elija una.
- Llamar a `ProductoRepository.buscarPorCategoria(id)` con el ID seleccionado.
- Mostrar los resultados con ID, nombre, precio y stock de cada producto.
- Si no hay productos en esa categoría, informarlo explícitamente.

4. Criterios de Evaluación

Item a evaluar	Descripción	Puntaje
HU-01: BaseRepository<T>	CRUD generico correcto, transacciones, Optional, cierre de EntityManager.	18 pts
HU-02: CategoriaRepo / ProductoRepo	Extension correcta, super() con Class<T>, buscarPorCategoria con JPQL.	12 pts
HU-03: Alta de categoria	Validacion de nombre, persistencia, ID visible.	8 pts
HU-04: Modificacion de categoria	Listado previo, error en ID invalido, campo vacio conserva valor.	10 pts
HU-05: Baja de categoria	Baja logica, error en ID invalido, no aparece en listados.	7 pts
HU-06: Alta de producto	Listado de categorias, validacion precio/stock, @ManyToOne resuelto.	12 pts
HU-07: Modificacion de producto	Valores actuales visibles, validaciones de precio y stock.	10 pts
HU-08: Baja de producto	Baja logica, mensaje con nombre, error en ID invalido.	8 pts
HU-09: Consulta JPQL	JPQL correcto, TypedQuery<Producto>, parametro nombrado, sin casteos.	15 pts

PUNTAJE TOTAL	100 puntos
Minimo para aprobar	60 puntos

5. Condiciones de Entrega

1. Entregar el proyecto Gradle completo comprimido en .zip con el nombre Apellido_Nombre_ParcialJPA.zip.
2. El proyecto debe compilar y ejecutar sin errores. Se descuentan 10 puntos por cada error de compilación.
3. No modificar ninguna clase del TP base: entidades, enums, JPAUtil ni persistence.xml.
4. Los 4 archivos nuevos (BaseRepository, CategoriaRepository, ProductoRepository y Main) deben estar en el paquete correcto.
5. Las opciones del menu deben ser accesibles desde el menu principal de consola.
6. El metodo buscarPorCategoria debe incluir un comentario explicando que hace la consulta JPQL.

Entrega

La entrega deberá incluir:

➤ 1. Repositorio

- El código debe estar en un único archivo .zip con nombre "Nombre_Alumno_parcial_2.zip" y dejar en los comentarios de la entrega el link al video.
- El proyecto debe ser funcional y ejecutable
- Debe incluir un archivo README.md con:
 - Descripción breve del proyecto
 - Instrucciones para ejecutarlo

2. Video de presentación (obligatorio)

Se deberá entregar un video con las siguientes características:

- ⌚ Duración: entre 10 y 15 minutos
- 📹 Cámara encendida durante toda la exposición
- 🔊 Audio claro y comprensible

Contenido del video

En el video deberás:

1. Presentarte brevemente
 - Motrar el funcionamiento de la aplicación:
 - Probar la generación de la Categoría.
 - Probar la creación de productos y asociarlos a categoría.
 - Mostrar los productos por id de categoría
 - Eliminar un producto de la categoría.
 - Mostrar los productos por id de categoría y comprobar que ya no se muestra.
2. Explicar:
 - Qué funcionalidades lograste implementar
 - Cómo abordaste la resolución
 - Qué dificultades encontraste (si las hubo) y cómo las resolviste

6. Historias de Usuario

6.1 Repositorios

HU-01 | Repositorio generico con CRUD

Como desarrollador

quiero contar con un BaseRepository<T> que implemente las operaciones CRUD comunes
para no repetir codigo de persistencia en cada repositorio especifico

Criterios de aceptacion

1. guardar() abre su propia transaccion, persiste con merge() y la cierra. Hace rollback ante error.
2. buscarPorId() retorna Optional<T>: Optional.of(entidad) si existe, Optional.empty() si no.
3. listarActivos() usa JPQL con WHERE e.eliminado = false y retorna List<T>.
4. eliminarLogico() busca por ID, establece eliminado = true, persiste y retorna true. Retorna false si no encuentra el registro.
5. Cada metodo cierra el EntityManager en un bloque finally.

Prioridad: Alta

Story Points: 18

HU-02 | Repositorios especificos de Categoria y Producto

Como desarrollador

quiero contar con CategoriaRepository y ProductoRepository que extiendan BaseRepository
para operar sobre cada entidad sin reescribir el CRUD base

Criterios de aceptacion

1. CategoriaRepository extiende BaseRepository<Categoria> y llama a super(Categoria.class).
2. ProductoRepository extiende BaseRepository<Producto> y llama a super(Producto.class).
3. ProductoRepository incluye buscarPorCategoria(Long categoriaId) con JPQL tipado.
4. La consulta JPQL filtra por categoria.id = :categoriaId y eliminado = false.
5. El metodo retorna List<Producto>.

Prioridad: Alta

Story Points: 12

6.2 Categorías

HU-03 | Dar de alta una categoría

Como operador del sistema
quiero poder crear una nueva categoría ingresando nombre y descripción
para organizar los productos del catálogo en grupos temáticos

Criterios de aceptación

1. El sistema solicita nombre y descripción por consola.
2. Si el nombre está vacío, el sistema informa el error y no persiste.
3. Al guardar exitosamente, se muestra el ID generado por la base de datos.
4. La categoría queda con `eliminado = false` y `createdAt` con la fecha/hora actual.

Prioridad: Alta

Story Points: 8

HU-04 | Modificar una categoría existente

Como operador del sistema
quiero poder editar el nombre o la descripción de una categoría ya creada
para corregir errores sin tener que borrar y recrear el registro

Criterios de aceptación

1. El sistema lista las categorías activas antes de pedir el ID.
2. Si el ID no corresponde a ninguna categoría activa, se muestra un mensaje de error.
3. Se muestran los valores actuales antes de pedir los nuevos.
4. Dejar un campo en blanco mantiene el valor anterior.
5. El cambio se persiste correctamente en la base de datos.

Prioridad: Alta

Story Points: 10

HU-05 | Dar de baja una categoría

Como operador del sistema
quiero poder dar de baja una categoría que ya no se utiliza
para ocultarla del sistema sin perder el historial de datos

Criterios de aceptación

1. La baja es lógica: se establece `eliminado = true`, el registro permanece en la BD.
2. Si el ID no existe o ya está dado de baja, el sistema informa el error.
3. La categoría dada de baja no aparece en ningún listado activo.
4. Se confirma la operación mostrando el nombre de la categoría afectada.

Prioridad: Media

Story Points: 7

6.3 Productos

HU-06 | Dar de alta un producto

Como operador del sistema
quiero poder registrar un nuevo producto asociandolo a una categoria existente
para incorporar articulos al catalogo con toda su informacion basica

Criterios de aceptacion

1. El sistema lista las categorias activas para que el operador seleccione una.
2. Si no hay categorias activas, se informa y se cancela la operacion.
3. Se solicitan: nombre, descripción, precio (mayor a 0) y stock (mayor o igual a 0).
4. Si precio o stock tienen valores invalidos, se informa el error y no se persiste.
5. Al guardar, se muestra el ID generado y la categoria asignada.

Prioridad: Alta

Story Points: 12

HU-07 | Modificar un producto

Como operador del sistema
quiero poder actualizar el nombre, precio y stock de un producto existente
para mantener el catalogo actualizado sin recrear el registro

Criterios de aceptacion

1. El sistema lista los productos activos antes de pedir el ID.
2. Si el ID no existe o el producto esta dado de baja, se muestra error.
3. Se muestran los valores actuales antes de pedir los nuevos.
4. Dejar un campo en blanco conserva el valor anterior.
5. Precio no puede actualizarse a un valor menor o igual a 0.
6. Stock no puede actualizarse a un valor negativo.

Prioridad: Alta

Story Points: 10

HU-08 | Dar de baja un producto

Como operador del sistema
quiero poder dar de baja un producto que ya no esta disponible
para retirarlo del catalogo activo sin eliminar su historial

Criterios de aceptacion

1. La baja es logica: eliminado = true, el registro permanece en la BD.
2. Si el ID no existe o ya esta dado de baja, se informa el error.
3. El producto dado de baja no aparece en el listado de productos activos.
4. Se muestra confirmacion con el nombre del producto afectado.

Prioridad: Media

Story Points: 8

6.4 Consulta JPQL

HU-09 | Listar productos de una categoria

Como operador del sistema
quiero poder ver todos los productos activos que pertenecen a una categoria especifica
para consultar el catalogo filtrado sin tener que revisar todos los productos

Criterios de aceptacion

1. El sistema lista las categorias activas para que el operador seleccione una.
2. La consulta esta implementada en ProductoRepository con JPQL y parametro nombrado :categoriald.
3. Se usa TypedQuery<Producto>; no hay casteos manuales en el codigo.
4. Solo se incluyen productos con eliminado = false.
5. El resultado muestra: ID, nombre, precio y stock de cada producto.
6. Si la categoria no tiene productos activos, se informa explicitamente.

Prioridad: Alta

Story Points: 15